Eightfold Engineering Intern (Jul-Dec 2026) - Assignment
Multi-Source Candidate Data Transformer

The Problem

Eightfold ingests candidate information from many places at once. Downstream products need one clean, canonical profile per candidate: a fixed set of fields, normalized formats, deduplicated across sources, and a record of where each value came from and how confident we are in it. Wrong-but-confident is worse than honestly-empty, because a bad value silently pollutes hiring decisions. Your job is to build the transformer that turns the messy inputs into that one trustworthy profile.

Inputs (Source Types)

The source types fall into two groups. **You must handle at least one source from each group** - structured source and unstructured source. Any source may be missing, empty, or malformed, and the same person may appear in several sources with conflicting values.

Structured sources — pick at least one

- **Recruiter CSV export** — structured rows (name, email, phone, current_company, title).
- **ATS JSON blob** — semi-structured, with its own field names that do NOT match ours.

Unstructured sources — pick at least one

- **GitHub profile URL** — a public REST/GraphQL API is available (name, bio, repos, languages).
- **LinkedIn profile URL** — profile fields (name, headline, experience, education).
- **Resume file(s)** — PDF / DOCX prose.
- **Recruiter notes (.txt)** — free text.

Default output schema

This is a starting point - yours to refine. The canonical profile has one fixed set of fields:

Field	Type / shape	Notes
candidate_id	string	
full_name	string	
emails	string[]	
phones	string[]	E.164 format
location	{ city, region, country }	country: ISO-3166 alpha-2
links	{ linkedin, github, portfolio, other[] }	
headline	string | null	
years_experience	number | null	
skills	[{ name, confidence, sources[] }]	canonical skill names
experience	[{ company, title, start, end, summary }]	dates as YYYY-MM
education	[{ institution, degree, field, end_year }]	
provenance	[{ field, source, method }]	where each value came from
overall_confidence	number	

Required twist — configurable output

On top of the default schema, your pipeline must accept a runtime config that reshapes the output — same engine, no code changes. The config can:

- Select a subset of fields to include.
- Rename / remap a field from a canonical path (the "from" key).
- Set per-field normalization (e.g. E.164 for phones, canonical for skills).
- Toggle provenance and confidence on or off.
- Choose what to do when a value is missing: null, omit, or error.

Keep a clean separation between your internal canonical record and a projection layer, and validate the result against the requested schema.

Example config:

```
{
  "fields": [
    { "path": "full_name", "type": "string", "required": true },
    { "path": "primary_email", "from": "emails[0]", "type": "string", "required": true },
    { "path": "phone", "from": "phones[0]", "type": "string", "normalize": "E164" },
    { "path": "skills", "from": "skills[].name", "type": "string[]", "normalize": "canonical" }
  ],
  "include_confidence": true,
  "on_missing": "null"
}
```

Input / output surface (UI or CLI)

Provide a thin way to feed inputs in and view the result — either a small **command-line tool** (point it at the input files + a config, print/write the JSON) or a **minimal UI** that shows the final profile. This is intentionally **lower priority**: a clean CLI is completely sufficient, and a basic input/output view is enough if you go the UI route. Don't spend your time on polish here — the engine, correctness, and reasoning matter far more. Mention in your README how to run whichever surface you build.

Constraints

- **Deterministic & explainable** — same inputs produce the same output; every field is traceable to a source and method.
- **Robust** — a missing or garbage source must not crash the run; unknown values become null, never invented.
- **Scale** — reasonable on thousands of candidates.

Step 1 - Technical Design · one page · no code

Before writing code, produce a single page that frames the problem and your plan. This is where we see how you think. Reference the problem statement above as needed.

Your one-pager should cover

- A pipeline / step breakdown (something like: detect → extract → normalize → merge → confidence → project-to-output → validate). This is just a dummy pipeline — be creative and build your own.
- Your canonical output schema and the normalized formats you chose (dates, phones, country, skill names, …).
- Your merge / conflict-resolution policy (match keys, how you pick a winner) and how you assign confidence.
- How you handle the runtime custom-output config (projection + validation).
- 3–5 edge cases and how you handle them, and what you would deliberately leave out under time pressure.

Deliverable:
A one-page document (PDF) named <YourFullName>_<YourEmail>_Eightfold.pdf

Step 2 - Implementation · working code

Now build it. Implement the design from Step 1. Work against the same problem statement and the sample inputs provided.

Your implementation should

- Run end-to-end on the provided sample inputs and emit schema-valid JSON for the default schema and at least one custom config.
- Cover at least 2 source types — **at least one structured and one unstructured** (one from each group above).
- Normalize correctly (dates, phones at minimum); skills are canonicalized.
- Merge across sources into one record, with provenance and confidence populated.
- Validate output before returning it; degrade gracefully on a missing/garbage source.
- Expose a thin input/output surface — a CLI or a minimal UI (lower priority; a CLI is fine).
- **[Optional]** Include a couple of tests or a gold-profile comparison — ideally one that covers an edge case.

Deliverable:
A runnable public GitHub repo link: source, a README with exact run steps, the produced output, and tests + a short demo video link (≈2 min, see Submission Guidelines). Note assumptions and anything descoped.

Submission Guidelines

Language & tools. Use any language and libraries you like. AI assistants are allowed but you own every line and must be able to explain and defend it. We're evaluating your judgment, not your typing speed.

What to submit: a one-page design document (PDF) + a runnable public GitHub repo with a short README (how to run it), the output it produced on the sample inputs, and your tests + a short demo video (see below).

Demo video. A short screen recording (about 2 minutes) in which you run the pipeline end-to-end on the sample inputs, show the default output and at least one custom-config output, and briefly talk through one design decision you're proud of and one edge case you handled. We use it to see the working system and hear you reason about it in your own words.

How we evaluate

Your submission is reviewed in two passes: first the Stage 1 design, then the Stage 2 project. We look for a correct, working core; the right approach for each part of the problem with clear reasoning; honest handling of edge cases and scope; and a design you can explain. A strong partial solution with sharp reasoning is a great outcome.

All the Best!